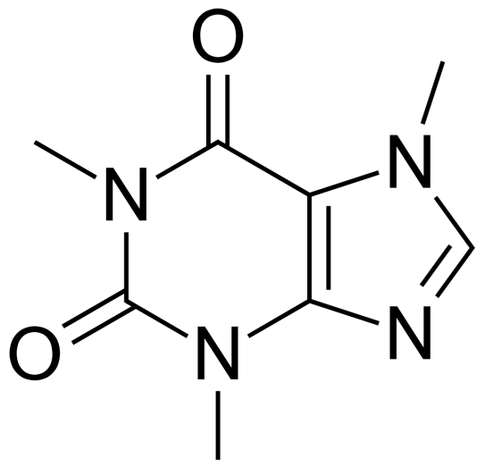
Graph Neural Networks

Lukas Galke Poech

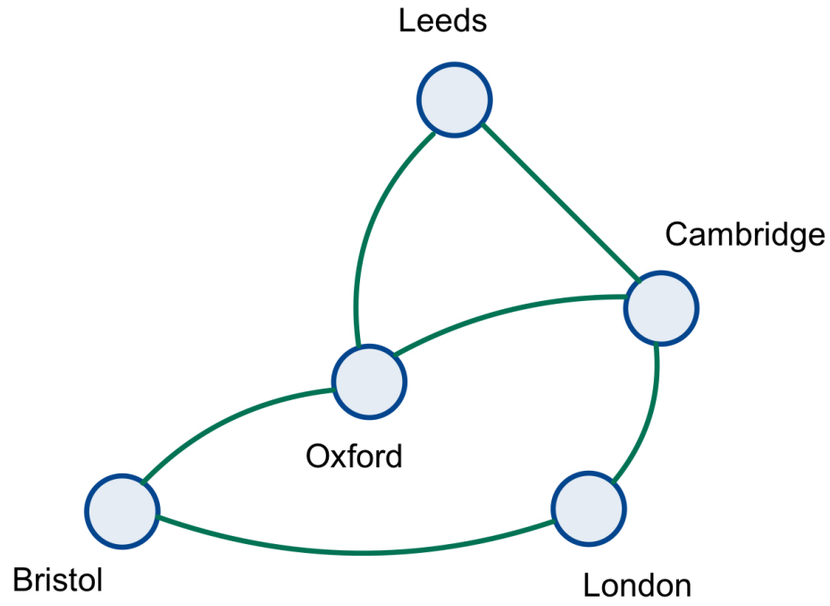
Keywords

- Neural message passing
- Graph neural networks
- Graph convolution
- Jumping knowledge
- Graph attention

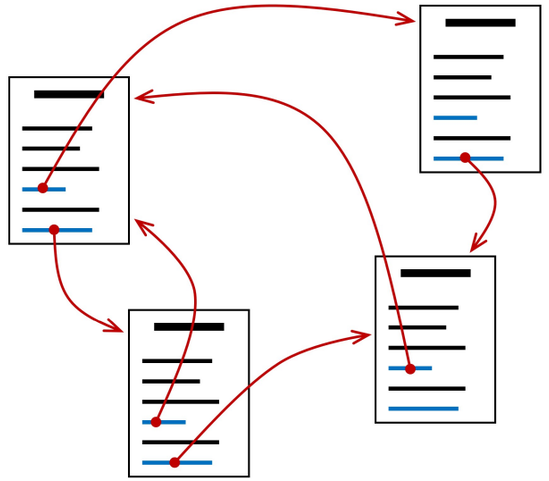
Graphs in the wild



(a)



(b)

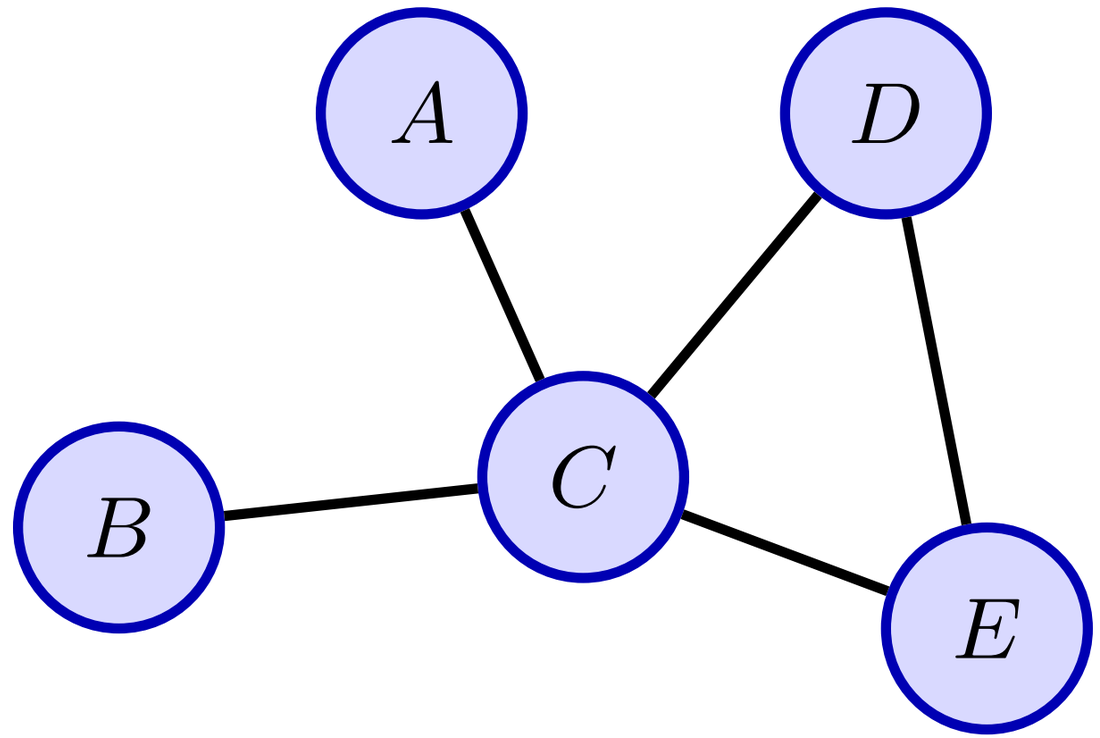


(c)

a) Molecules, b) Traffic networks, c) Citation networks, ...

Graphs

- Sets of nodes V
- and edges $E \subseteq V \times V$
- Node features: $V \to \mathbb{R}^d$



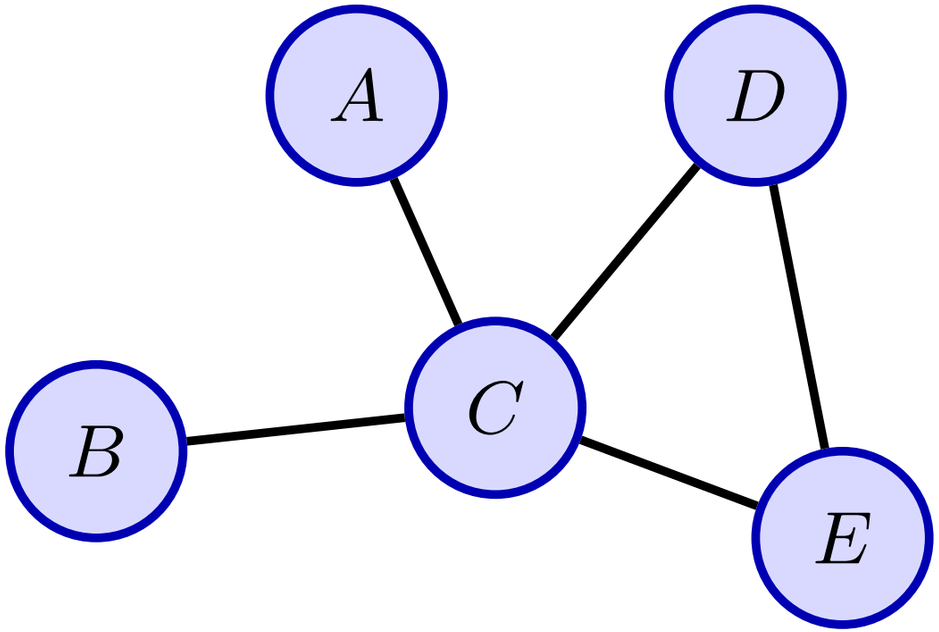
Common tasks on graphs

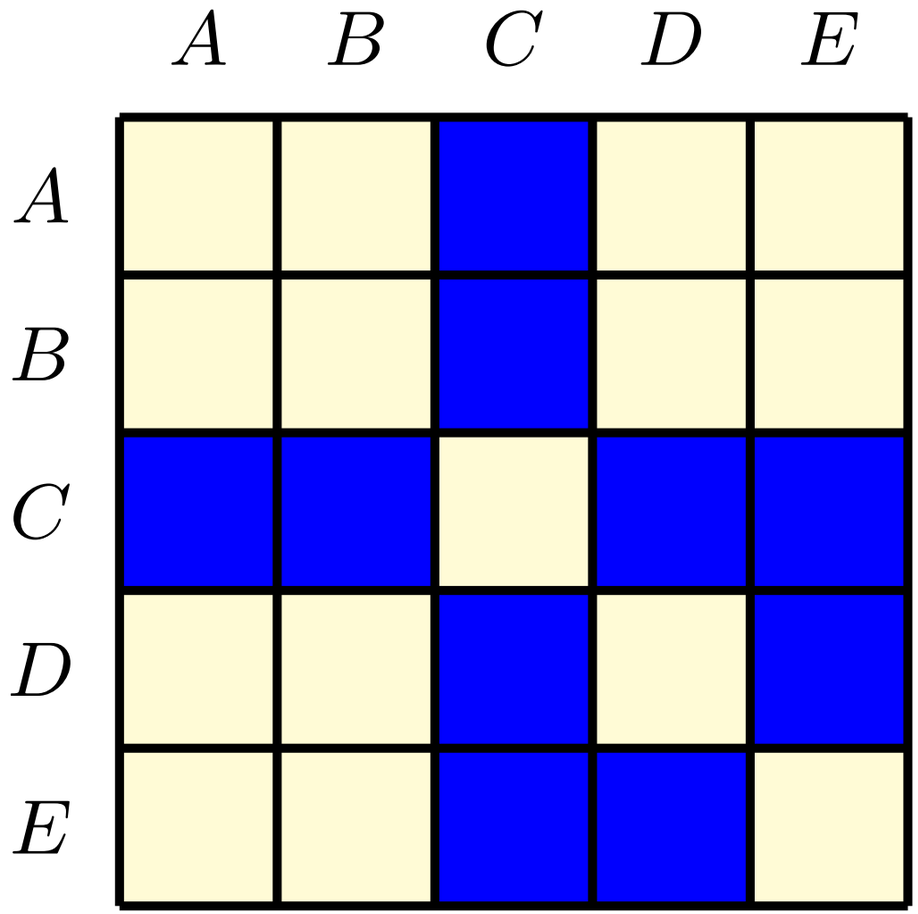
Node classification: Predict the class of each node.

Graph classification: Predict the class of an entire graph.

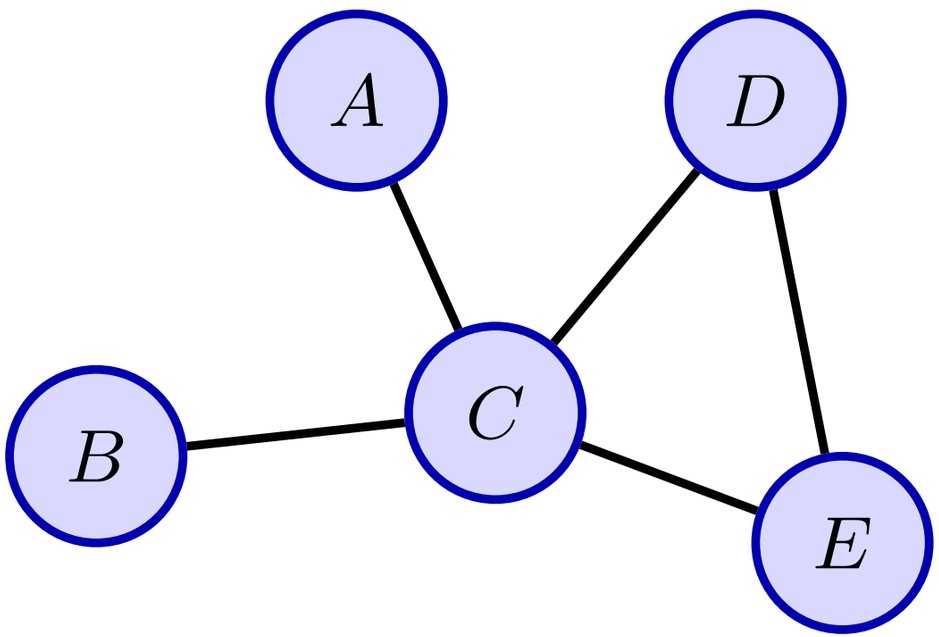
Link prediction: Predict the (future) existence of an edge.

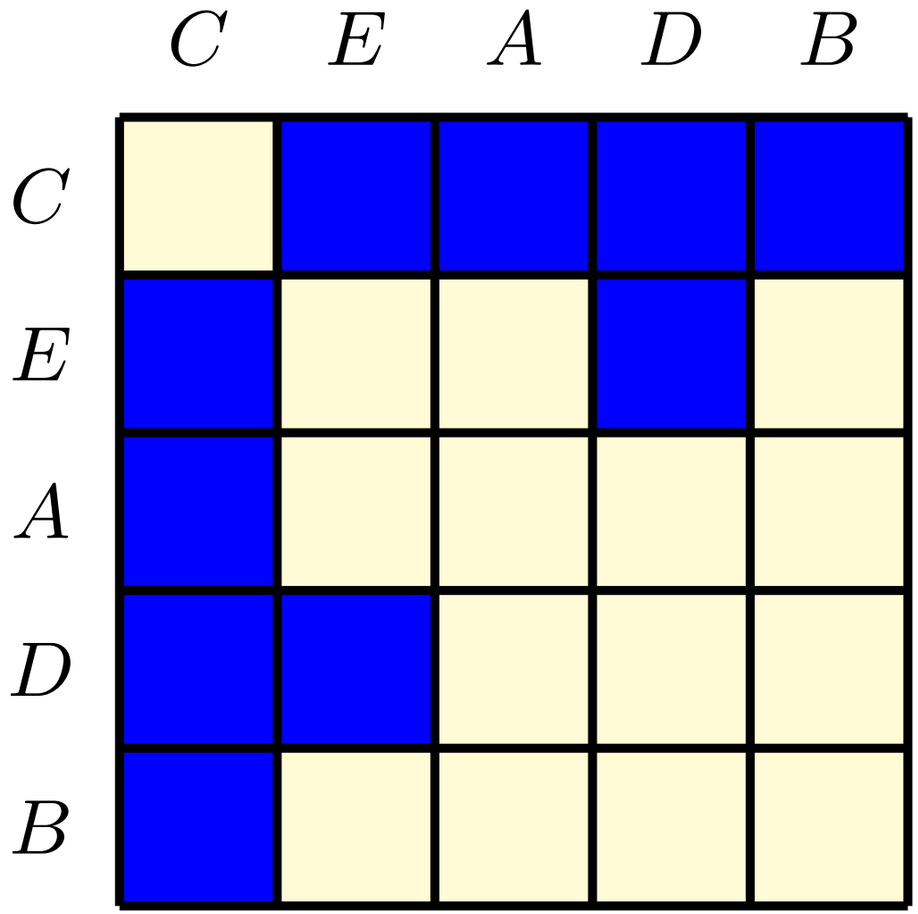
The adjacency matrix





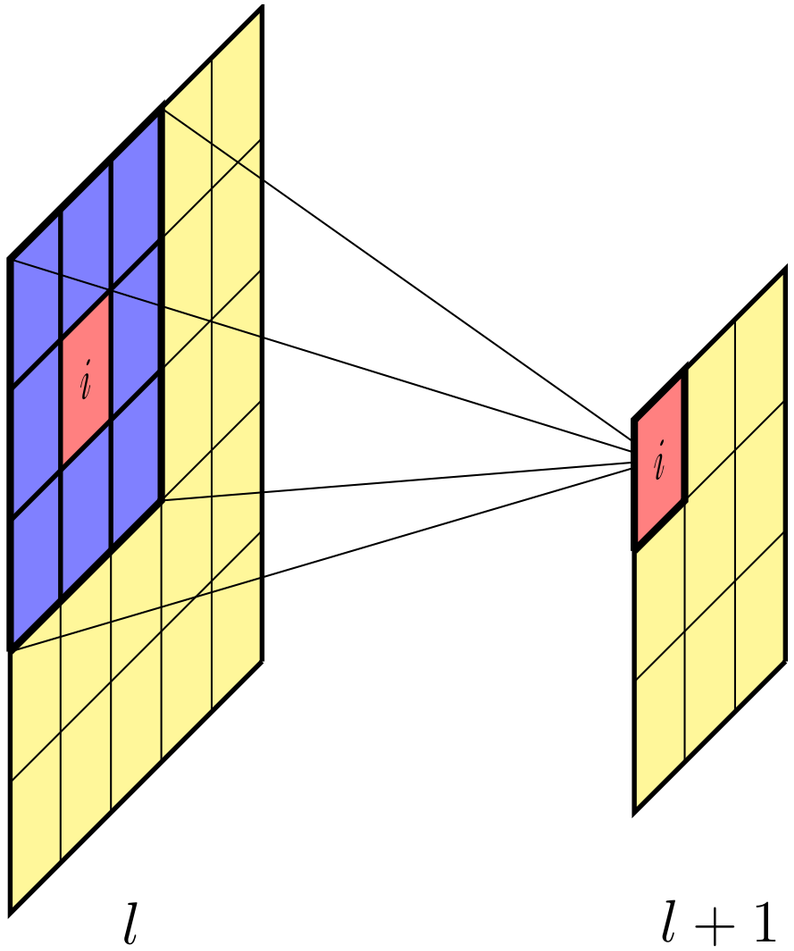
Permutation invariance

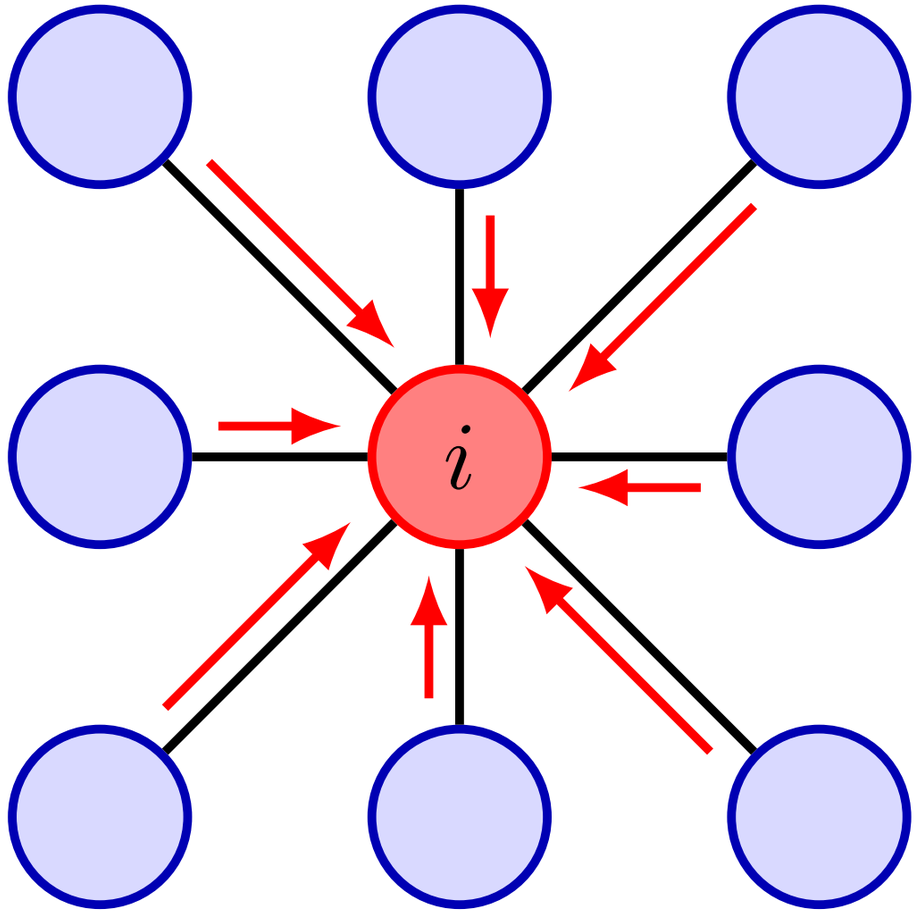




How to process graph-structured data?

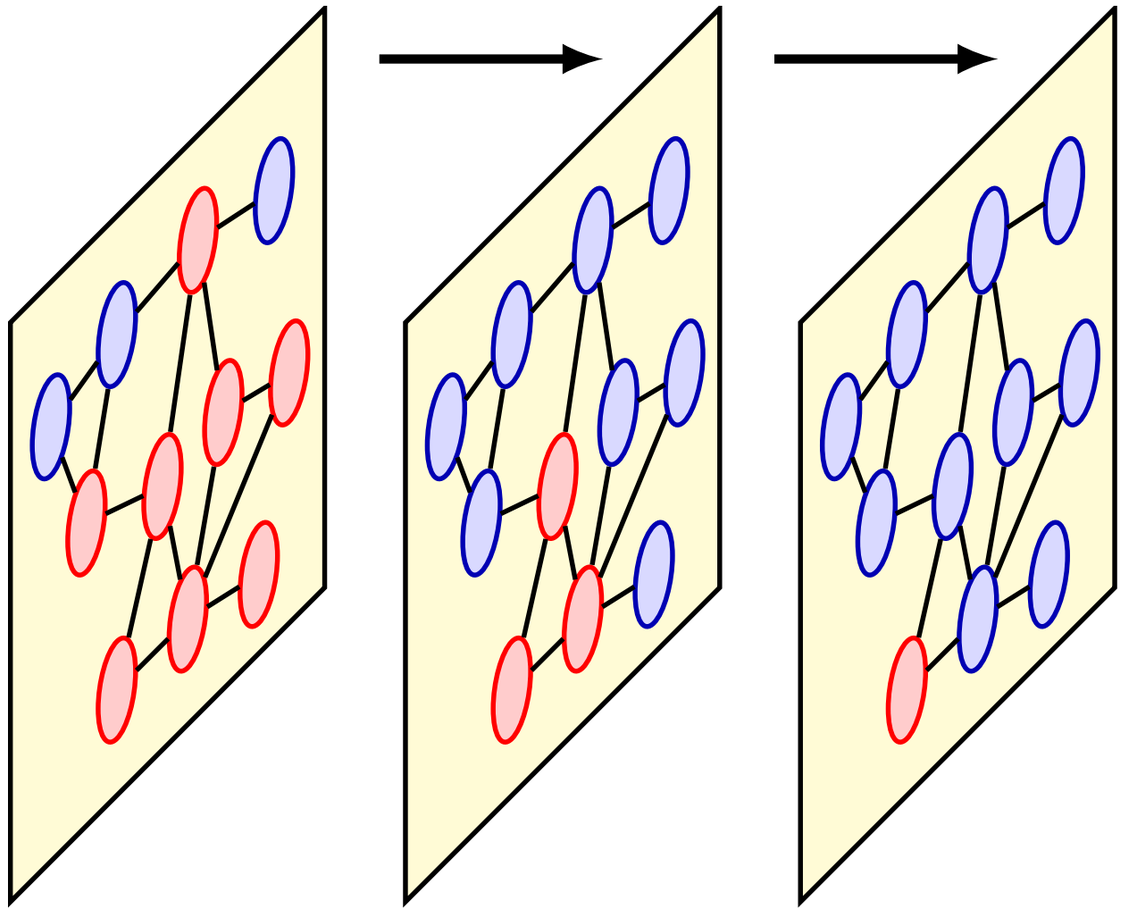
Convolution for Graphs?





Neighborhood Aggregation

... through repeated convolution along the graph structure



Neural message passing

A generalized formulation based on two operations:

AGGREGATE: A method to aggregate messages from all neighbors.

UPDATE: Update each node representation based on aggregated messages.

Neural message passing (cont.)

Set $\mathbf{h_u}^{(0)}$ to the input features of node u.

$$\mathbf{h_u}^{(k+1)} = \mathrm{UPDATE}\left(\mathbf{h_u}^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right)$$

where $\mathcal{N}(u)$ gives us a set of all neighbor nodes of node u and $\mathbf{m}_{\mathcal{N}(u)}^{(k)}$ are the aggregated messages:

$$\mathbf{m}_{\mathcal{N}(u)}^{(k)} = \mathrm{AGGREGATE}^{(k)}\left(\left\{\mathbf{h_v}^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right)$$

Neural message passing (cont.)

After running K iterations of message passing, we can use the output of the final layer to define the embeddings for each node, i.e.,

$$\mathbf{z_u} = \mathbf{h_u}^{(K)}, \forall u \in V.$$

The basic graph neural network

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)} \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right)$$

The basic graph neural network

$$\text{UPDATE}\left(\mathbf{h_u}^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right) = \mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)}\mathbf{m}_{\mathcal{N}(u)}^{(k)} + \mathbf{b}^{(k)}$$

$$\mathbf{m}_{\mathcal{N}(u)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{\mathbf{h_v}^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right) = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)}$$

Neighborhood normalization

Graph *convolutional* networks use a symmetric normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h_v}}{\sqrt{|\mathcal{N}(u)| \cdot |\mathcal{N}(v)|}}$$

Alternatively, use the number of in-going edges for normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h_v}}{|\mathcal{N}(u)|}$$

Graph convolution (Kipf & Welling, 2017)

A specific, very popular form of graph neural network. Here in matrix notation:

- Nodes features $\mathbf{X} = \mathbf{H}^{(0)}$
- Adjacency matrix $\mathbf{A}$

$$\mathbf{H}^{(\mathbf{k}+\mathbf{1})} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(\mathbf{k})}\mathbf{W}^{(\mathbf{k})} + b\right)$$

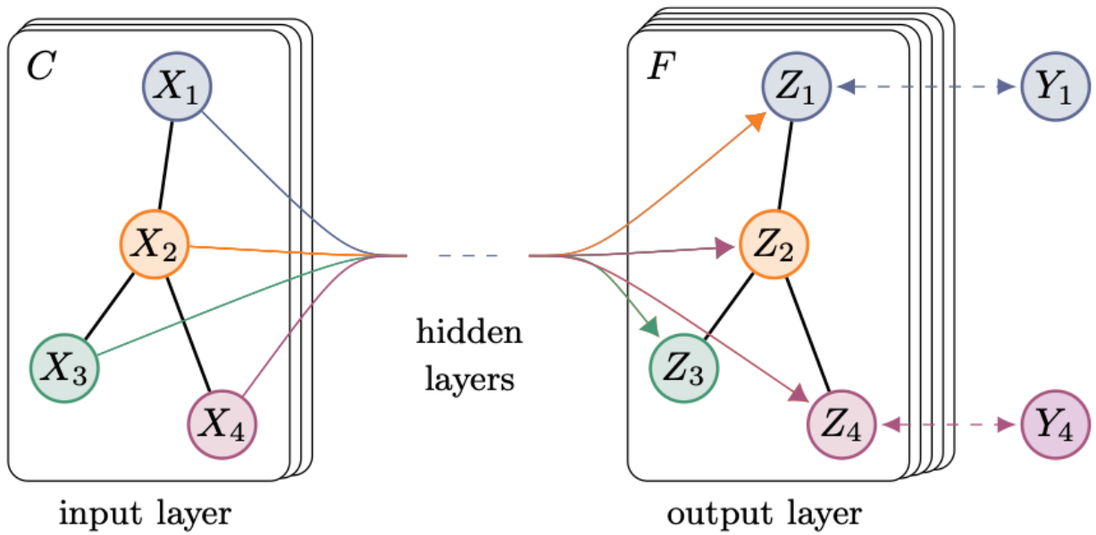
where $\hat{\mathbf{A}}$ is a normalized version of the adjacency matrix, with *self-loops* added.

Task: Node Classification

Task: Classify each node, given node features $\mathbf{X}$ and adjacency matrix $\mathbf{A}$.

Key assumption: Nodes that are connected, are more likely to belong to the same class.

$$\mathbf{Z} = f(\mathbf{X}, \mathbf{A}) = \operatorname{softmax}\left(\hat{\mathbf{A}}\,\operatorname{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\right)$$



Task: Graph Classification

For graph-level tasks, we need to aggregate the entire graph.

Key strategy: **Pooling** after GNN layers, often called **READOUT** operation.

$$\mathrm{READOUT}(\mathbf{h}^{(k)}) = \sum_{u \in V} \mathbf{h_u}^{(k)}$$

where $h^{(k)}$ are the node representations after applying k GNN layers.

Then, apply a final linear layer or MLP on the **READOUT** result.

$$z = \mathrm{MLP}\left(\mathrm{READOUT}(\mathbf{h}^{(k)})\right)$$

Keywords

- **Neural message passing**
- **Graph neural networks**
- **Graph convolution**
- Jumping knowledge
- Graph attention

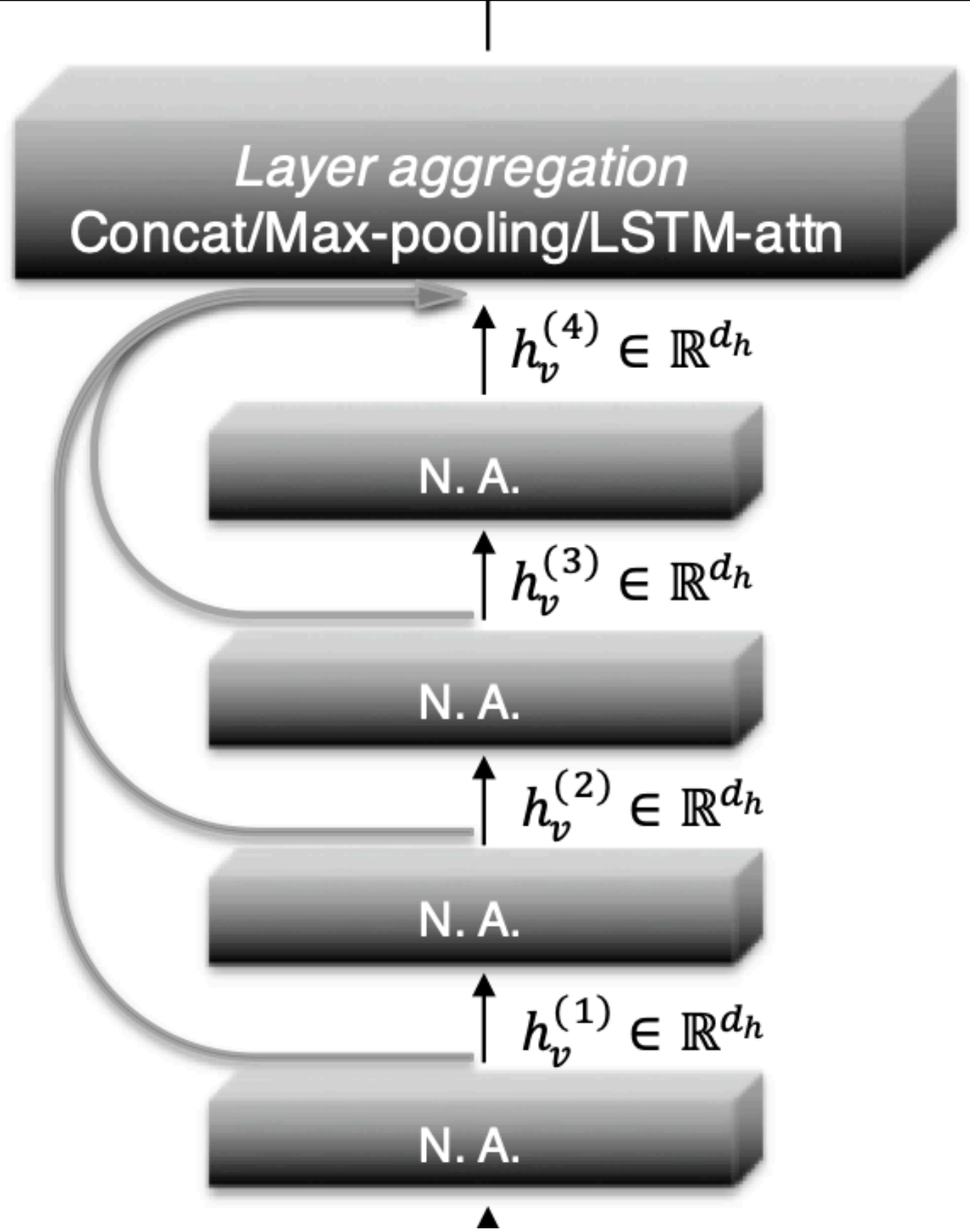
Jumping knowledge

Oversmoothing is a common problem when stacking too many layers, due to many consecutive ~averaging operations.

What helps? Insert a connection from each layer's output to the final layer, called **jumping knowledge**:

$$\mathbf{z} = \mathrm{MLP}\left(\mathbf{h}^{(1)} \oplus \mathbf{h}^{(2)} \oplus \cdots \oplus \mathbf{h}^{(k)}\right)$$

where $\cdot \oplus \cdot$ is concatenation.



Graph attention (GAT)

In basic graph neural networks, all edges are considered to have equal weight.

But some edges may be more important than others.

Graph attention: learn to predict the weights of the edges with an attention mechanism:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h_u}$$

where $\alpha_{u,v} = \mathbf{softmax}_{\mathcal{N}(u)} \left(\mathbf{a}_T \left[\mathbf{W h_u} \oplus \mathbf{W h_v} \right] \right)$ with trainable vector $\mathbf{a}$.

Summary

- **Graph neural networks** enable us to make use of neural networks for tasks on graph-structured data.
- **Neural message passing** is the general framework to specify graph neural networks through **UPDATE** and **AGGREGATE** operations.
- **Graph convolution** performs convolution along graph structure
- **Jumping knowledge**, i.e., shortcut connections to the final layer, helps to avoid oversmoothing (similar to residual connections).
- **Graph attention** predicts edge weights from participating nodes.
- Graph-level tasks require a global **pooling** mechanism.

Keywords

- Neural message passing
- Graph neural networks
- Graph convolution
- Jumping knowledge
- Graph attention

Further reading:

Graph Representation Learning book by Will Hamilton

How's my teaching

admonymous.co/lukasgalke